

CHAPTER 9 • DAY 1 OF 2

Operating System Support

Processes, scheduling, and how the OS stays in control



Hardware and Software, Stacked



Three Interfaces That Matter



ISA

Instruction Set Architecture. The boundary between hardware and software: the machine instructions a processor understands. The OS reaches a wider, privileged subset than an application does.



ABI

Application Binary Interface. Defines binary portability: the system call interface plus the hardware resources a compiled program can reach through the user ISA.



API

Application Programming Interface. The user ISA plus high-level library calls. Recompiling against the same API is what lets source code move between systems.

Operating System Services



Program creation

editors, debuggers, utility programs



Program execution

loading, resource setup



I/O device access

hides device specifics, devices have their own ISA



Controlled file access

format and permissions



System access

protects shared resources (authorization and contention/conflicts)



Error detection

contains, reports, recovers errors



Accounting

usage stats, tuning, and other information (could be use for billing purposes)

The most important system program: it masks hardware detail and gives every program a consistent interface to work through.

The OS as Resource Manager

A computer is a set of resources for the movement, storage, and processing of data, and for controlling those functions. The OS manages those resources.



It is unusual as a control mechanism in two respects.



(1) It runs the same way as any other software: a program executed by the processor.



(2) It frequently relinquishes control, then depends on the processor to hand control back.

RESOURCES UNDER MANAGEMENT



Processor time



Main memory



I/O devices



Files and programs

Types of Operating Systems



Interactive

The user works directly with the computer, often communicating with it during the run itself.



Batch

Jobs are grouped and submitted together; the user sees results only after the whole run finishes.



Uniprogramming

One program occupies the processor at a time, start to finish, before the next begins.



Multiprogramming

Several programs sit in memory at once; the processor switches among them to stay busy.

Early Systems, Late 1940s to Mid-1950s

The programmer worked the console directly: switches, lights, a reader, a printer. No OS stood in between. Two problems fell out of that.



Scheduling, by sign-up sheet

Users reserved blocks of processor time on paper. Finish early and the machine sat idle. Run long and you were cut off before you were done.



Setup time

Loading the compiler, loading the source program, saving the compiled output, then loading and linking it again, all before a single job actually ran.

The Resident Monitor

the resident monitor

Interrupt processing

Device drivers

Job sequencing

Control language interpreter

User program area

Always resident in memory, the monitor reads in one job at a time and hands control to it. When the job ends or errors, control returns to the monitor, which loads the next job with no idle gap in between.

JOB CONTROL LANGUAGE

A small language the user includes with a job so the monitor knows what to do: which compiler to load, when to run, where the data starts. Lines like \$JOB, \$FTN, \$LOAD, and \$RUN are instructions to the monitor, not to the program.

Desirable Hardware Features



Privileged instructions

Only the monitor may execute them. A user program that tries triggers an error interrupt. I/O instructions are privileged, so the monitor keeps control of every device.



Memory protection

A user program must not alter the memory holding the monitor. Hardware detects the attempt and hands control to the monitor, which aborts the job.



Timer

Prevents one job from monopolizing the system. When it expires, an interrupt fires and control returns to the monitor.

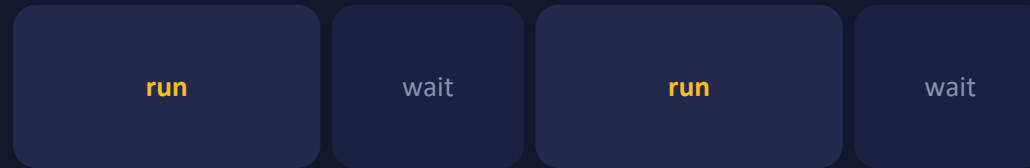


Interrupts

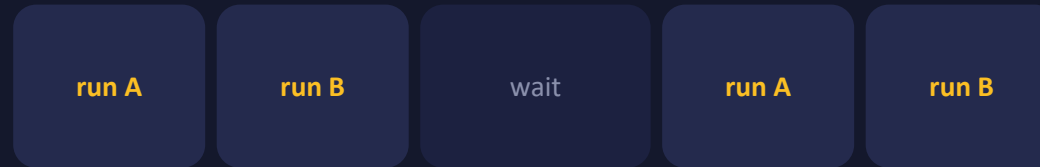
Gives the OS a flexible way to relinquish control to a user program and reliably get it back.

Multiprogramming

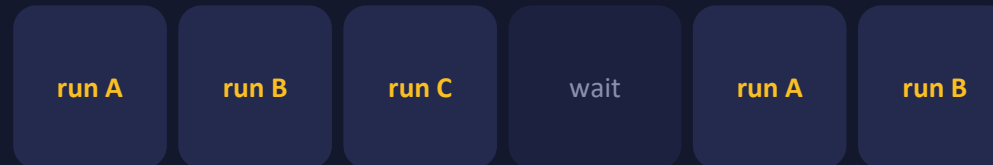
Uniprogramming



Multiprogramming, 2 jobs



Multiprogramming, 3 jobs



Idle time on one job becomes run time for another. Room for two or three programs in memory keeps the processor working instead of waiting on I/O.

Time Sharing Systems

When multiprogramming is used to serve interactive users rather than batch jobs, we call it time sharing. The processor's time is sliced into short quanta and shared among everyone connected.



Multiple users, multiple terminals, one processor interleaving all of them in short bursts.

THE MATH BEHIND IT

n users
actively requesting service

$$1 / n$$

is roughly the share of effective computer speed each user sees, before OS overhead.

TABLE 9.3

Batch Multiprogramming vs. Time Sharing



	Batch Multiprogramming	Time Sharing
Principal objective	Maximize processor use	Minimize response time
Directives to the OS come from	Job control language, submitted with the job	Commands typed at the terminal

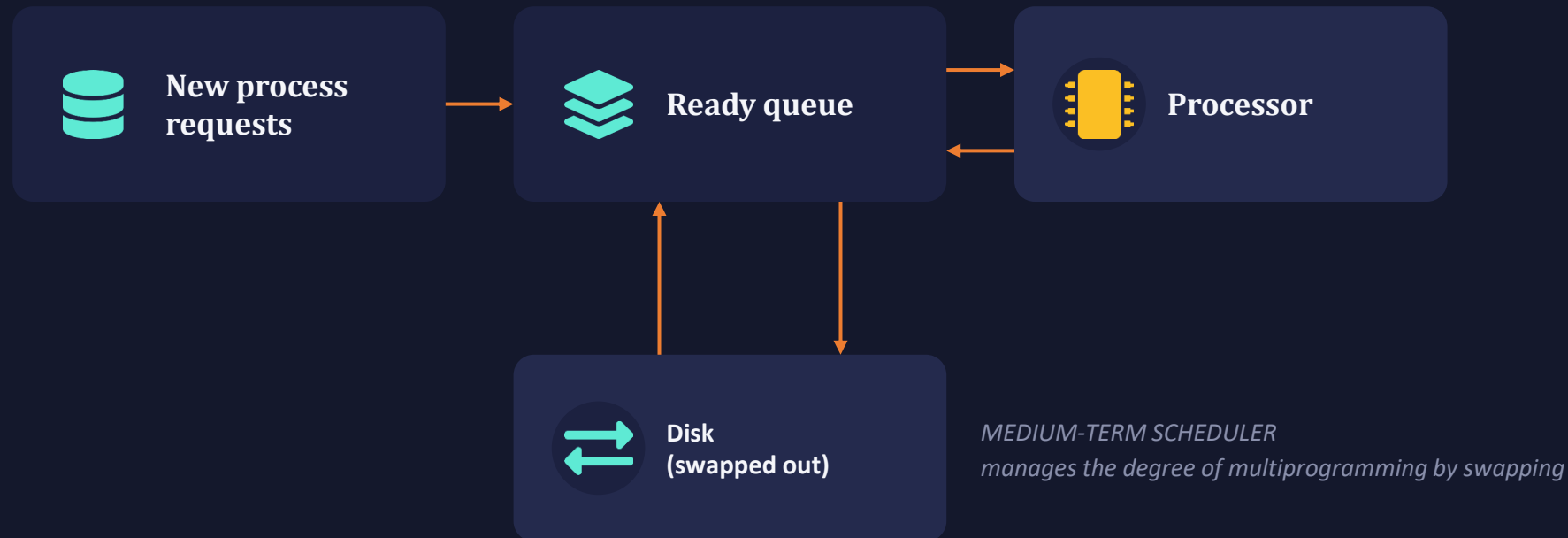
How Scheduling Works

LONG-TERM SCHEDULER

admits a job, decides which and when

SHORT-TERM SCHEDULER (DISPATCHER)

fine-grained, runs frequently, picks what runs next



A fourth type, I/O scheduling, decides which pending I/O request an available device handles next.

FIGURE 9.7

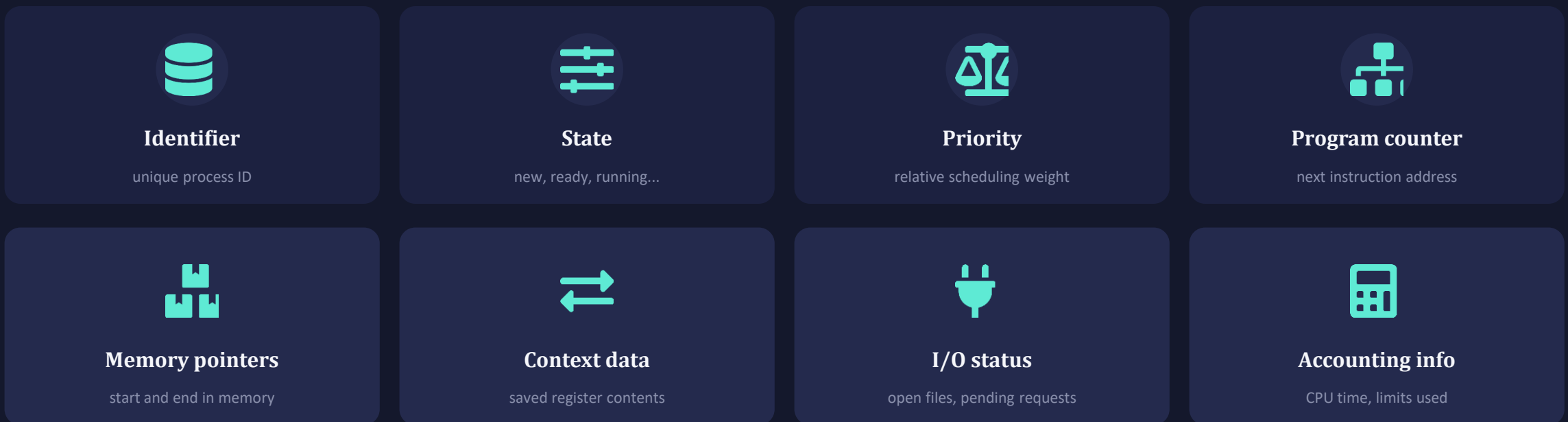
The Five-State Process Model



FIGURE 9.8

The Process Control Block

One PCB per process. It is how the OS remembers everything about a process the moment it stops running, so it can pick up exactly where it left off later.



When Memory Runs Out of Room

Multiprogramming helps, but the processor is so much faster than I/O that every process in memory can end up waiting at once. Swapping is one answer: move a waiting process out to disk and bring in one that is ready to run.

WITHOUT SWAPPING



Long-term queue on disk feeds the OS directly. If everything in memory is waiting on I/O, the processor simply idles.

WITH SWAPPING



An intermediate queue on disk holds processes temporarily pushed out of memory, so the OS always has something ready to run next.



LIVE ON THE DEMO MACHINE

Watching Process States and the PCB

ps, /proc, and a few processes we start on purpose

Where We Landed Today



The OS as manager

A program like any other, but one that controls every other program's access to hardware.



Multiprogramming

Several processes in memory so the processor never has to sit idle waiting on I/O.



Five process states

New, ready, running, waiting, halted, tracked for every process in its PCB.

Next class: what happens when memory itself runs out of room, partitioning, paging, and virtual memory, with a heavier hands-on stretch built in.